

Tema 3 - Funciones en Python

Francisco Gortázar

Funciones

Definición

► Sintaxis:

```
def my_function(arg1, arg2, arg3):  
    # Código de la función
```

Definición

► Ejemplo:

```
def fibonacci(num_terminos):  
    a = 0  
    b = 1  
    while num_terminos > 0:  
        print(a)  
        a = b  
        b = a + b  
        num_terminos -= 1
```

Llamado

► Sintaxis

```
my_function(arg1, arg2)
```

► Ejemplo:

```
fibonacci(10)
```

Retorno de Valores

► Sintaxis

```
def my_function(arg1, arg2):  
    return arg1 + arg2
```

► Ejemplo

```
def fibonacci(num_terminos):  
    a = 0  
    b = 1  
    while num_terminos > 0:  
        a = b  
        b = a + b  
        num_terminos -= 1  
    return a
```

► Llamada

```
value = fibonacci(10)  
print(value)
```

Parámetros nombrados

- ▶ Python permite pasar argumentos a una función indicando explícitamente el nombre del parámetro al que se asignan
- ▶ Flexibilidad: Al usar este método, el orden de los argumentos no importa.
- ▶ Claridad: Mejora la comprensión del código al saber exactamente qué valor corresponde a cada parámetro en la definición.

```
value = fibonacci(num_terminos=10)
print(value)
```

Parámetros por defecto

- ▶ Es posible asignar un valor predeterminado a un parámetro en la definición de la función.
- ▶ Funcionamiento: Si al llamar a la función se omite ese argumento, se utilizará automáticamente el valor por defecto.
- ▶ Regla de sintaxis: Los parámetros con valores por defecto deben listarse después de los parámetros que no los tienen para que los argumentos posicionales sigan funcionando correctamente.

```
def fibonacci(num_terminos=10):  
    a = 0  
    b = 1  
    while num_terminos > 0:  
        a = b  
        b = a + b  
        num_terminos -= 1  
    return a
```

```
value = fibonacci()
```


Parámetros opcionales

- ▶ Para que un argumento sea opcional, se le suele asignar el valor especial `None` por defecto en la definición
- ▶ En el cuerpo de la función, se utiliza una estructura condicional (`if`) para comprobar si se proporcionó un valor o si el parámetro permanece como `None`
- ▶ Útil cuando se desea que una función maneje información extra solo si el usuario decide enviarla

```
def fibonacci(num_terminos=None):  
    if num_terminos is None:  
        num_terminos = 10  
    a = 0  
    b = 1  
    while num_terminos > 0:  
        a = b  
        b = a + b  
        num_terminos -= 1  
    return a
```

Paso de funciones a funciones

- ▶ Las funciones son ciudadanos de primer orden
 - ▶ Pueden pasarse como argumentos
 - ▶ Pueden asignarse a variables

```
def starts_with_a(w):  
    return w.startswith("a")  
  
li = ["apple", "banana", "avocado", "cherry", "apricot"]  
res = filter(starts_a, li)  
print(list(res))
```

Funciones lambda

- ▶ Son funciones anónimas y se definen con la palabra clave `lambda`
- ▶ Restricción
 - ▶ Están limitadas a contener una única expresión en lugar de múltiples sentencias de código
- ▶ Uso
 - ▶ Se utilizan frecuentemente en programación funcional para operaciones rápidas que se pueden pasar como objetos a otras funciones

```
li = ["apple", "banana", "avocado", "cherry", "apricot"]  
res = filter(lambda w: w.startswith("a"), li)  
print(list(res))
```

```
l = ["apple", "banana", "avocado", "cherry", "apricot"]  
l.sort(key=lambda w: len(w))  
print(l)
```